

EC200U&EG91xU Series

QuecOpen(SDK)

I2C Development Guide

LTE Standard Module Series

Version: 1.1

Date: 2023-11-17

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Revision	Date	Author	Description
-	2021-11-09	Chaos HUANG	Creation of the document
1.0	2021-12-07	Chaos HUANG	First official release
1.1	2023-11-17	Kevin WANG	1. Updated the document name based on the unified QuecOpen naming. 2. Added applicable modules EG912U-GL and EG915U series. 3. Updated the I2C bus number enumeration: ql_i2c_channel_e (Chapter 2.3.1.1). 4. Updated the I2C error codes enumeration: ql_errcode_i2c_e (Chapter 2.3.1.3). 5. Updated the log capture tool description (Chapter 3.2).

Contents

About the Document.....	3
Contents	4
Table Index.....	5
Figure Index	6
1 Introduction	7
2 I2C API.....	8
2.1. Header File	8
2.2. API Overview.....	8
2.3. API Description.....	9
2.3.1. ql_I2cInit.....	9
2.3.1.1. ql_i2c_channel_e.....	9
2.3.1.2. ql_i2c_mode_e	10
2.3.1.3. ql_errcode_i2c_e	10
2.3.2. ql_I2cWrite	11
2.3.3. ql_I2cRead	12
2.3.4. ql_I2cRelease	12
2.3.5. ql_I2cWrite_16bit_addr	13
2.3.6. ql_I2cRead_16bit_addr.....	13
3 Example	15
3.1. I2C Development Example in Application Layer	15
3.2. I2C Feature Debugging	16
4 Appendix References	17

Table Index

Table 1: API Overview 8

Table 2: Related Documents 17

Table 3: Terms and Abbreviations 17

Figure Index

Figure 1: Create I2C Task and Enable I2C Example	15
Figure 2: Select Register Bit for I2C Slave Device	16

1 Introduction

Quectel EC200U and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces I2C development in application, including I2C API, I2C development example and feature debugging on Quectel EC200U and EG91xU family modules in QuecOpen® solution.

2 I2C API

2.1. Header File

ql_i2c.h, the header file of I2C API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all the header files mentioned in this document are in this directory.

2.2. API Overview

Table 1: API Overview

Function	Description
<i>ql_i2cInit()</i>	Initializes I2C bus.
<i>ql_i2cWrite()</i>	Writes the data to I2C bus. Length of the register address of slave device is 8 bits.
<i>ql_i2cRead()</i>	Reads the data from I2C bus. Length of the register address of slave device is 8 bits.
<i>ql_i2cRelease()</i>	Releases I2C bus.
<i>ql_i2cWrite_16bit_addr()</i>	Writes the data to I2C bus. Length of the register address of slave device is 16 bits.
<i>ql_i2cRead_16bit_addr()</i>	Reads the data from I2C bus. Length of the register address of slave device is 16 bits.

2.3. API Description

2.3.1. ql_I2cInit

This function initializes I2C bus.

- **Prototype**

```
ql_errcode_i2c_e ql_I2cInit(ql_i2c_channel_e i2c_no, ql_i2c_mode_e Mode)
```

- **Parameter**

i2c_no:

[In] I2C bus number. See **Chapter 2.3.1.1** for details.

Mode:

[In] The operating mode of I2C. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.3**.

2.3.1.1. ql_i2c_channel_e

The enumeration of I2C bus number:

```
typedef enum
{
    i2c_1 = 0,
    i2c_2,
    i2c_3,
}ql_i2c_channel_e;
```

- **Member**

Member	Description
<i>i2c_1</i>	The number of I2C bus is 1.
<i>i2c_2</i>	The number of I2C bus is 2.
<i>i2c_3</i>	The number of I2C bus is 3.

NOTE

In the QuecOpen solution, the EC200U series and EG91xU family modules support two I2C interfaces, and the number of I2C bus is 1 or 2.

2.3.1.2. ql_i2c_mode_e

The enumeration of I2C operating mode:

```
typedef enum
{
    STANDARD_MODE = 0,
    FAST_MODE = 1,
} ql_i2c_mode_e;
```

- **Member**

Member	Description
<i>STANDARD_MODE</i>	Standard mode with the transmission speed of 100 kbps.
<i>FAST_MODE</i>	Fast mode with the transmission speed of 400 kbps.

2.3.1.3. ql_errcode_i2c_e

The error codes of I2C indicate whether the function is successfully executed or not. The enumeration of error codes:

```
typedef enum
{
    QL_I2C_SUCCESS = QL_SUCCESS,

    QL_I2C_INIT_ERR                = 1|QL_I2C_ERRCODE_BASE,
    QL_I2C_NOT_INIT_ERR,
    QL_I2C_INVALID_PARAM_ERR,
    QL_I2C_WRITE_ERR               = 5|QL_I2C_ERRCODE_BASE,
    QL_I2C_READ_ERR,
    QL_I2C_RELEASE_ERR,
    QL_I2C_CREATE_MUTEX_ERR,
    QL_I2C_MUTEX_TIMEOUT_ERR
}ql_errcode_i2c_e;
```

● Member

Member	Description
<code>QL_I2C_SUCCESS</code>	Success execution.
<code>QL_I2C_INIT_ERR</code>	Fail to initialize I2C.
<code>QL_I2C_NOT_INIT_ERR</code>	I2C is not initialized.
<code>QL_I2C_INVALID_PARAM_ERR</code>	Invalid parameter(s).
<code>QL_I2C_WRITE_ERR</code>	Fail to write the data to I2C.
<code>QL_I2C_READ_ERR</code>	Fail to read the data from I2C.
<code>QL_I2C_RELEASE_ERR</code>	Fail to release I2C bus.
<code>QL_I2C_CREATE_MUTEX_ERR</code>	Failed to create mutex by kernel.
<code>QL_I2C_MUTEX_TIMEOUT_ERR</code>	Failed to obtain mutex by kernel.

2.3.2. ql_I2cWrite

This function writes the data to I2C. Length of the register address of slave device is 8 bits.

● Prototype

```
ql_errcode_i2c_e ql_I2cWrite(ql_i2c_channel_e i2c_no, uint8_t slave, uint8_t addr, uint8_t *data,
uint32_t length)
```

● Parameter

i2c_no:

[In] I2C bus number. See **Chapter 2.3.1.1** for details.

slave:

[In] Address of the I2C slave device.

addr:

[In] Register address of the I2C slave device.

data:

[In] The data to be written.

length:

[In] Length of the data to be written. Unit: byte.

- **Return Value**

See **Chapter 2.3.1.3**.

2.3.3. `ql_I2cRead`

This function reads the data from I2C bus. Length of the register address of slave device is 8 bits.

- **Prototype**

```
ql_errcode_i2c_e ql_I2cRead(ql_i2c_channel_e i2c_no, uint8_t slave, uint8_t addr, uint8_t *buf,
uint32_t length)
```

- **Parameter**

i2c_no:

[In] I2C bus number. See **Chapter 2.3.1.1** for details.

slave:

[In] Address of the I2C slave device.

addr:

[In] Register address of the I2C slave device.

buf:

[Out] The data read from I2C bus.

length:

[In] Length of the data to be read from I2C bus. Unit: byte.

- **Return Value**

See **Chapter 2.3.1.3**.

2.3.4. `ql_I2cRelease`

This function releases I2C bus. If the same I2C bus needs to be re-initialized, this function should be called to release it before calling `ql_I2cInit()` for initialization.

- **Prototype**

```
ql_errcode_i2c_e ql_I2cRelease(ql_i2c_channel_e i2c_no)
```

- **Parameter**

i2c_no:

[In] I2C bus number. See **Chapter 2.3.1.1** for details.

- **Return Value**

See **Chapter 2.3.1.3**.

2.3.5. ql_I2cWrite_16bit_addr

This function writes the data to I2C bus. Length of the register address of slave device is 16 bits.

- **Prototype**

```
ql_errcode_i2c_e ql_I2cWrite_16bit_addr(ql_i2c_channel_e i2c_no, uint8_t slave, uint16_t addr, uint8_t *data, uint32_t length)
```

- **Parameter**

i2c_no:

[In] I2C bus number. See **Chapter 2.3.1.1** for details.

slave:

[In] Address of the I2C slave device.

addr:

[In] Register address of the I2C slave device.

data:

[In] The data to be written.

length:

[In] Length of the data to be written. Unit: byte.

- **Return Value**

See **Chapter 2.3.1.3**.

2.3.6. ql_I2cRead_16bit_addr

This function reads the data from I2C bus. Length of the register address of slave device is 16 bits.

- **Prototype**

```
ql_errcode_i2c_e ql_I2cRead_16bit_addr(ql_i2c_channel_e i2c_no, uint8_t slave, uint16_t addr, uint8_t *buf, uint32_t length)
```

- **Parameter**

i2c_no:

[In] I2C bus number. See **Chapter 2.3.1.1** for details.

slave:

[In] Address of the I2C slave device.

addr:

[In] Register address of the I2C slave device.

buf:

[Out] The data read from I2C bus.

length:

[In] Length of the data to be read from I2C bus. Unit: byte.

- **Return Value**

See **Chapter 2.3.1.3**.

3 Example

This chapter introduces how to use the above APIs in application layer for I2C development and simple debugging.

3.1. I2C Development Example in Application Layer

The I2C example file *I2C_demo.c*, provided in the CSDK of EC200U and EG91xU family modules, is in the *components\ql-application\i2c* directory. Since I2C needs a slave device for communication, in the example, the GC032A camera is used for feature demonstration by default. The API in the file is parsed as below:

- *ql_i2c_demo_init()*: Create a task. This function needs to be called before enabling I2C example.
- *ql_i2c_demo_thread()*: The execution function of the task for initializing I2C, reading data from I2C and writing data to I2C.

To enable *I2C_demo.c*, you need to have *ql_i2c_demo_init()* called in *ql_init_demo_thread()* and the GC032A camera connected to the module. An example is shown below:

```

269:
270: static void ql_init_demo_thread(void *param)
271: {
272:     QL_INIT_LOG("init demo thread enter, param 0x%x", param);
273:
274:     /*Caution:If the macro of secure boot and the function are opened, download firmware and restart will enable secure boot.
275:       the secret key cannot be changed forever*/
276:     #ifdef QL_APP_FEATURE_SECURE_BOOT
277:         //ql_dev_enable_secure_boot();
278:     #endif
279:
280:     #if 0
281:         ql_gpio_app_init();
282:         ql_gpioint_app_init();
283:     #endif
284:
285:     #ifdef QL_APP_FEATURE_I2C
286:         /*because I2C uses the camera as a demonstration, so i2c demo can not be opened when the camera demo open */
287:         ql_i2c_demo_init();
288:     #endif
289:
290:
291:     #ifdef QL_APP_FEATURE_LEDCFG
292:         //ql_ledcfg_app_init();
293:     #endif
294:
295:     #ifdef QL_APP_FEATURE_AUDIO
296:         //ql_audio_app_init();
297:     #endif
298:     #ifdef QL_APP_FEATURE_HEADSET_DET
299:         //ql_headset_det_app_init();
300:     #endif

```

Figure 1: Create I2C Task and Enable I2C Example

Register address of the I2C slave device is 1 byte (8 bits) by default. If **demo_for_8bit_or_16bit** is set to 1, the default length is 1 byte. If the I2C slave device whose register address is 2 bytes (16 bits) is used, **demo_for_8bit_or_16bit** should be set to 0.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#include "ql_api_osi.h"
#include "ql_log.h"
#include "ql_api_camera.h"
#include "ql_i2c.h"
#include "I2C_demo.h"

/*=====
 * Macro Definition
 *=====*/

#define QL_APP_I2C_LOG_LEVEL          QL_LOG_LEVEL_INFO
#define QL_APP_I2C_LOG(msg, ...)      QL_LOG(QL_APP_I2C_LOG_LEVEL, "QL_APP_I2C", msg, ##__VA_ARGS__)
#define QL_APP_I2C_LOG_PUSH(msg, ...) QL_LOG_PUSH(QL_APP_I2C, msg, ##__VA_ARGS__)

#define QL_I2C_TASK_STACK_SIZE      1024
#define QL_I2C_TASK_PRIO            APP_PRIORITY_NORMAL
#define QL_I2C_TASK_EVENT_CNT      5

#define SalveAddr_w_8bit             (0x42 >> 1)
#define SalveAddr_r_8bit             (0x43 >> 1)

/*reserved*/
#define SalveAddr_w_16bit            (0xff >> 1)
#define SalveAddr_r_16bit            (0xff >> 1)

#define demo_for_8bit_or_16bit (1) //1:test 8bit register address 0:test 16bit register address
/*=====
 * Struct
 *=====*/

/*=====
 * Enum
 *=====*/
```

Figure 2: Select Register Bit for I2C Slave Device

3.2. I2C Feature Debugging

The I2C feature of EC200U and EG91xU family modules can be debugged through Quectel LTE OPEN EVB.

The debug procedure is shown below:

1. Download the compiled firmware to the module.
2. Connect the USB port of the LTE OPEN EVB to the PC with a USB cable.
3. Capture the log with the log capture tool “cooltools” via USB AP Log Port. For details about log capture, see **document [2]**.
4. The USB AP Log Port mainly displays the system debugging information. After you capture the log, you will be able to check the development related information.

4 Appendix References

Table 2: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
AP	Access Point
EVB	Evaluation Board
IoT	Internet of Things
I2C	Inter-Integrated Circuit
RTOS	Real-Time Operating System
SDK	Software Development Kit